

Making figures with pub_style

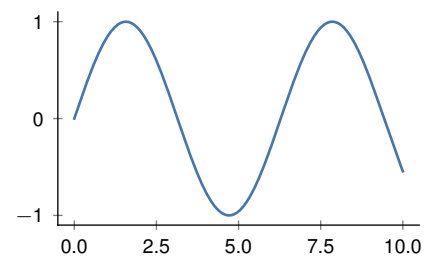
Abdullah Mustafa

June 13, 2026

I got tired of looking at horrible matplotlib fonts, so I wrote pub_style.

```
1 import numpy as np
2 import pub_style
3
4 X = np.linspace(0, 10, 1000)
5 Y = np.sin(X)
6
7 pub_style.use()
8 fig, ax = pub_style.new_figure(width_mm=56)
9 ax.plot(X, Y)
10 fig.savefig("plots-gallery/simplest.pdf")
```

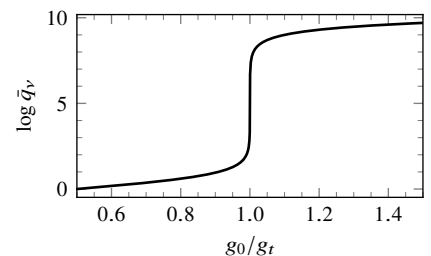
```
1 import numpy as np
2 import pub_style
3
4 y = np.linspace(0.5, 1.5, 1000)
5
6 q_sat = 1e10
7
8 q = 0.5 * q_sat * (y - 1) + 0.5 * q_sat * np.sqrt((y - 1) ** 2
9     + 4 * y / q_sat)
10
11 pub_style.use(serif=True)
12 fig, ax = pub_style.new_figure(despine=False, minor_ticks=True,
13     width_mm=56)
14
15 ax.set_xlabel(r'$g_0 / g_t$')
16 ax.set_ylabel(r'$\log \bar{q}_{\nu}$')
17 ax.set_xlim(0.5, 1.5)
18 ax.plot(y, np.log10(q), 'k')
19 fig.savefig('plots-gallery/laserAtThreshold')
```

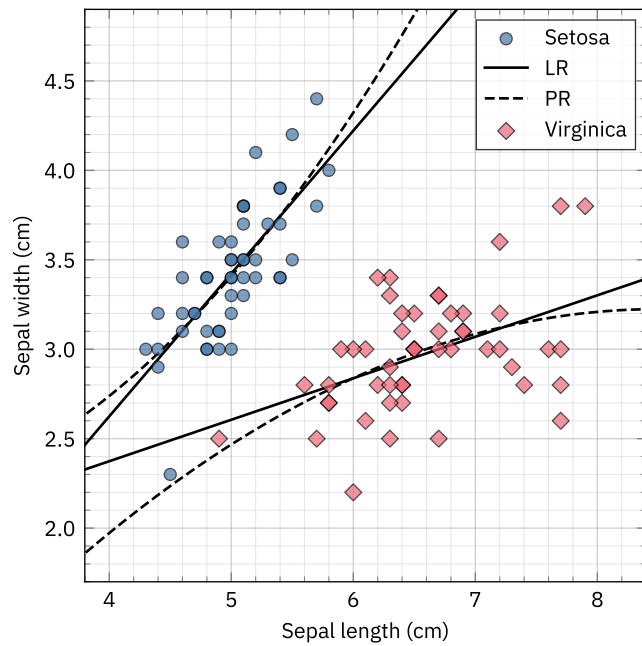


That gets you this figure.

That gets you this figure.

```
1 import numpy as np
2 import pub_style
3 from sklearn.linear_model import LinearRegression
4 from sklearn.preprocessing import PolynomialFeatures
5 from sklearn.datasets import load_iris
6
7 iris_data = load_iris()
8 X_data = iris_data.data
9 y_target = iris_data.target
10
11 feature_names = iris_data.feature_names
12 target_names = iris_data.target_names
```





```

13
14 x_col = 0
15 y_col = 1
16
17 polynomial_degree = 2
18
19 datasets_to_plot = ["setosa", "virginica"]
20
21 pub_style.use(False)
22 pub_style.use_font('IBM Plex Sans')
23 fig, ax = pub_style.new_figure(minor_ticks=True, despine=False
24                               , ratio=1)
25
26 ax.set_xlabel(f"{feature_names[x_col].capitalize()}")
27 ax.set_ylabel(f"{feature_names[y_col].capitalize()}")
28
29 # ax.grid(True, which="major", linestyle="-", linewidth=0.5,
30 #         alpha=0.6)
31 # ax.grid(True, which="minor", linestyle="-", linewidth=0.4,
32 #         alpha=0.3)
33 # ax.set_axisbelow(True)
34 pub_style.griddify() # griddify does all of the above!
35
36 markers = ['o', 's', 'D']
37
38 # # And the colour for the lines (regressions):
39 # regression_color = "#8a8a8a" # <-- neutral grey
40 regression_color = 'k'
41 # regression_color = '#7f9fa1' # <-- greyish seafoam
42
43 # # Plotting in a loop for each dataset:
44 all_data_x_to_plot = []
45 all_data_y_to_plot = []
46 regression_flag = 0 # control plotting of regression labels
47 for class_name in datasets_to_plot:

```

```

45 class_index = list(target_names).index(class_name) # Get
    correct label
46 class_mask = y_target == class_index
47 data_x = X_data[class_mask, x_col].reshape(-1, 1)
48 data_y = X_data[class_mask, y_col]
49
50 # Scatter plot:
51 ax.scatter(
52     data_x,
53     data_y,
54     label=class_name.capitalize(),
55     s=20,
56     # color = dataset_colors[class_index],
57     # edgecolors=dataset_line_colors[class_index],
58     edgecolors='k',
59     marker=markers[class_index],
60     # linewidths = 1.5,
61     zorder=3,
62     alpha=0.7,
63 )
64
65 # Linear regression:
66 lin_model = LinearRegression().fit(data_x, data_y)
67 x_range = np.linspace(data_x.min() - 2.0, data_x.max() +
    2.0, 100).reshape(-1, 1)
68 y_pred_linear = lin_model.predict(x_range)
69 ax.plot(
70     x_range,
71     y_pred_linear,
72     label="LR" if regression_flag == 0 else "_nolegend_",
73     # linewidth = 2.6,
74     color=regression_color,
75     zorder=2, # use the z-order to force scatter to be
        displayed over lines
76 )
77
78 # Polynomial regression:
79 poly = PolynomialFeatures(polynomial_degree)
80 data_x_poly = poly.fit_transform(data_x)
81 poly_model = LinearRegression().fit(data_x_poly, data_y)
82 x_range_poly = poly.transform(x_range)
83 y_pred_poly = poly_model.predict(x_range_poly)
84 ax.plot(
85     x_range,
86     y_pred_poly,
87     label="PR" if regression_flag == 0 else "_nolegend_",
88     # linewidth = 2.6,
89     color=regression_color,
90     linestyle="--",
91     zorder=2,
92 )
93
94 # Tracking which data we plot to adjust plotting limits
    later:
95 all_data_x_to_plot.extend(data_x)
96 all_data_y_to_plot.extend(data_y)
97
98 regression_flag += 1
99

```

```
100 # After the loop, replace ax.margins(0) with:
101 x_min, x_max = np.min(all_data_x_to_plot), np.max(
    all_data_x_to_plot)
102 y_min, y_max = np.min(all_data_y_to_plot), np.max(
    all_data_y_to_plot)
103
104 ax.set_xlim(x_min - 0.5, x_max + 0.5)
105 ax.set_ylim(y_min - 0.5, y_max + 0.5)
106 ax.legend()
107
108 fig.savefig("plots-gallery/regresfigure.svg")
```